

Time Series Event Detection in R: Harbinger Tutorial Workflow



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>



Learning Roadmap

A guided learning path for event detection in time series using the **harbinger** R package. Each tutorial builds on the last — from your very first detection to advanced motif analysis — so your understanding grows naturally and cumulatively.

Getting Started

T01: Minimal workflow · T02: Exploring the data · T03: Plotting time series

Anomaly Detection

T04: Histogram-based detector · T05: ARIMA detector · T06: Evaluating detections

Change-Points & Transformations

T07: AMOC detector · T08: Moving averages (MAS) · T09: SAX & XSAX

Motifs & Patterns

T10: Motifs and discords

📄 **Try it yourself!** Complete examples with generated plots are available in the official repository. Run each tutorial's code and follow along with the visual results at: <https://github.com/cefet-rj-dal/harbinger/tree/master/examples>

Primary reference: Ogasawara, E., Salles, R., Porto, F., Pacitti, E. *Event Detection in Time Series*. Springer, 2025. doi:10.1007/978-3-031-75941-3

The Harbinger Framework

Load Data

Import your time series dataset.

Configure Model

Set algorithm parameters.

`fit()`

Train model on historical data.

`detect()`

Identify events in the series.

`evaluate()`

Assess detection performance.

`har_plot()`

Visualize detected events.

Structured Output

`detect()` always returns a data frame with three columns: `idx` (index), `event` (logical), and `type` (event category).

Unified Contract

`fit()`, `detect()`, and `evaluate()` share the same interface — swap any algorithm without changing downstream code.

Universal Visualization

`har_plot()` works identically for anomalies, change-points, and motifs — one interface for all event types.

Getting Started

```
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple
head(dataset)

## serie event
## 1 1.0000000 FALSE
## 2 0.9689124 FALSE

# Plot series with labels
har_plot(harbinger(),
  dataset$serie,
  event = dataset$event)
```

```
# Create default model
model <- harbinger()

# Run detection
detection <- detect(model, dataset$serie)
head(detection)

## idx event type
## 1 1 FALSE
## 2 2 FALSE

# Plot detections vs labels
har_plot(model, dataset$serie,
  detection, dataset$event)
```

Step by Step

- Load the package and `examples_anomalies$simple`
- Inspect `head()`: columns `serie` and `event`
- Plot the series with `har_plot()` before any model
- Create the default `harbinger()` object
- Run `detect()` and explore the output
- Compare detections against ground-truth labels

📄 **Key concept:** A Harbinger method receives a time series and returns a standardized, event-oriented output — ready to plot or evaluate.

Understanding Your Data

```
library(harbinger)

# Helper reutilizável
dataset_summary <- function(x) {
  first_series <- x[[1]]
  meta_cols <- c("idx", "event", "type",
                "seq", "seqlen")
  signal_cols <- setdiff(
    names(first_series), meta_cols)
  dataset_type <- if (
    "value" %in% names(first_series) ||
    length(signal_cols) == 1
  ) "univariate" else "multivariate"
  plot_column <- if (
    "value" %in% names(first_series)
  ) "value" else signal_cols[1]
  list(n_series = length(x),
       dataset_type = dataset_type,
       signal_cols = signal_cols,
       plot_column = plot_column,
       first_series = first_series)
}
```

```
data(A1Benchmark)
A1Benchmark <- loadfulldata(A1Benchmark)
info <- dataset_summary(A1Benchmark)

info$n_series    ## [1] 67
info$dataset_type ## [1] "univariate"
info$signal_cols ## [1] "value"

har_plot(harbinger(),
  info$first_series[[info$plot_column]],
  event = info$first_series$event)
```

Step by step

- Define `dataset_summary()` to inspect any collection at a glance
- Load `A1Benchmark` and expand with `loadfulldata()`
- Confirm: 67 series, univariate, single value column
- Plot the first signal with its event labels
- Run the helper before committing to a detection method

📌 **Key concept:** Knowing your data's shape — univariate vs. multivariate, labeling density — is the first step to choosing the right detection method.

Visualizing Time Series

```
library(harbinger)

# Univariate series
data(examples_anomalies)
uni <- examples_anomalies$simple
har_plot(harbinger(),
  uni$serie,
  event = uni$event)
```

```
# Multivariate series
data(oil_3w_Type_1)
oil_3w_Type_1 <- loadfulldata(oil_3w_Type_1)
first_mv <- oil_3w_Type_1[[1]]

meta_cols <- c("idx", "event", "type",
  "seq", "seqlen")

signal_cols <- setdiff(
  names(first_mv), meta_cols)

signal_cols
## [1] "p_pdg"  "p_tpt"
## [3] "t_tpt"  "p_mon_ckp"
## [5] "t_jus_ckp" "p_jus_ckgl"
## [7] "qgl"

# Plot one column from the multivariate dataset
har_plot(
  harbinger(),
  first_mv[[signal_cols[1]]],
  event = first_mv$event
)
```

Step by Step

- Use `har_plot()` for both univariate and multivariate series
- Load multivariate data with `loadfulldata()`
- Exclude metadata fields to isolate signal columns
- Select a single column to visualize with the same interface

📌 **Key concept:** `har_plot()` is the go-to diagnostic tool — visual inspection should precede any model selection.

Anomaly Detection in Time Series

Point Anomaly

Isolated spike deviating from surrounding values.

Contextual Anomaly

Unusual given local temporal context, not globally.

Collective Anomaly

Unusual subsequence where individual points seem normal.

Distribution-Based

Flags low-density values via histograms — `hanr_histogram()`.

Model-Based

Flags residuals exceeding a threshold — `hanr_arima()`.

Evaluation via confusion matrix: **Precision**, **Recall**, and **F1-score**, all accessible through the unified `detect()` interface.

Reference: Ogasawara, E., Salles, R., Porto, F., Pacitti, E. *Event Detection in Time Series*. Springer, 2025. doi:10.1007/978-3-031-75941-3

Your First Anomaly Detector

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple

# Plot the original series
har_plot(harbinger(),
         dataset$serie,
         event = dataset$event)

# Configure a histogram-based detector
model <- hanr_histogram(
  density_threshold = 0.05)
```

```
# Run detection
detection <- detect(model, dataset$serie)
head(detection)
## idx event type
## 1  1 TRUE anomaly
## 2  2 FALSE

# Evaluate against ground truth
evaluation <- evaluate(model,
  detection$event, dataset$event)
evaluation$confMatrix
##      event
## detection TRUE FALSE
##    TRUE   1    1
##   FALSE   0   99

# Plot detections alongside labels
har_plot(model, dataset$serie,
         detection, dataset$event)
```

Step by step

- Load `daltoolbox` and `harbinger`
- Configure `hanr_histogram(density_threshold = 0.05)`
- Run `detect()` — low-density bins are flagged
- Evaluate with `evaluate()`: 2x2 confusion matrix
- Result: 1 TP, 1 FP, 0 FN, 99 TN
- Plot to compare detections vs. ground truth

📌 **Key concept:** Distribution-based detection flags rare values relative to the empirical distribution — an excellent baseline before trying more complex methods.

Model-Based Detection with ARIMA

```
library(daltoolbox)
library(harbinger)
data(examples_anomalies)
dataset <- examples_anomalies$simple

# Configure and fit the ARIMA detector
model <- hanr_arma()
model <- fit(model, dataset$serie)

# Run detection
detection <- detect(model, dataset$serie)

# Evaluate
evaluation <- evaluate(model,
  detection$event, dataset$event)
evaluation$confMatrix
##      event
## detection TRUE FALSE
##   TRUE    1    0
##   FALSE    0   100
```

```
# Plot detections on the original series
har_plot(model, dataset$serie,
  detection, dataset$event)

# Plot the residual signal with threshold
har_plot(
  model,
  attr(detection, "res"),
  detection,
  dataset$event,
  yline = attr(detection, "threshold")
)
```

Step by Step

- Create `hanr_arma()` and `fit()` to the series
- Run `detect()` on the time series
- Perfect result: 1 TP, 0 FP, 0 FN, 100 TN
- Plot residuals via `attr(detection, "res")` with threshold
- Residuals reveal *why* each point was flagged

📌 **Key concept:** ARIMA models expected temporal structure; anomalies are points that deviate sharply from predictions — shifting detection from "rare values" to "unexpected behavior."

Evaluating Detections

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple
model <- hanr_arima()
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
```

```
# Strict evaluation (hard)
hard_eval <- har_eval()
hard_result <- evaluate(hard_eval,
  detection$event, dataset$event)
hard_result$confMatrix
##      event
## detection TRUE FALSE
##   TRUE    1    0
##   FALSE    0   100
```

```
# Tolerant evaluation (soft)
soft_eval <- har_eval_soft()
soft_result <- evaluate(soft_eval,
  detection$event, dataset$event)
soft_result$confMatrix
##      event
## detection TRUE FALSE
##   TRUE    1    0
##   FALSE    0   100
```

```
# Metrics returned by both:
# accuracy=1, F1=1, precision=1,
# recall=1, balanced_accuracy=1
```

Step by step

- Fit `hanr_arima()` and run `detect()`
- **Hard eval** (`har_eval()`): requires exact timestamp match
- **Soft eval** (`har_eval_soft()`): tolerates small temporal shifts
- Both return accuracy, F1, precision, recall, `balanced_accuracy`
- Results agree here; may diverge on real-world data

📌 **Key concept:** Soft evaluation rewards detections that are close but not exact — acknowledging that near-miss timing can still be meaningful in practice.

Change-Point Detection

Point Anomaly

An isolated data point deviating from the expected pattern.

Change-Point

A persistent, abrupt shift in the underlying statistical properties of a series.

Change-points mark a persistent regime shift — not just a spike.

At Most One Change (AMOC)

- **Goal:** Find the single best split separating two statistically distinct segments
- **Method:** Scans every possible split position and selects the one minimizing an objective function (e.g., sum of squared residuals)
- **Harbinger:** `hcp_amoc()` returns a data frame with `event=TRUE` and `type="changepoint"` at the detected index
- **Applications:** Fault detection in industrial systems; regime shifts in financial data

Reference: Ogasawara, E., Salles, R., Porto, F., Pacitti, E. *Event Detection in Time Series*. Springer, 2025. doi:10.1007/978-3-031-75941-3

Your First Change-Point Detector

```
library(harbinger)

data(examples_changepoints)
dataset <- examples_changepoints$simple
head(dataset)
## serie event
## 1 0.00 FALSE
## 2 0.25 FALSE

# Plot the series with the labeled change-
# point
har_plot(harbinger(),
         dataset$serie,
         event = dataset$event)
```

```
# Configure the AMOC detector
model <- hcp_amoc()

# Run detection
detection <- detect(model, dataset$serie)

# Result: change-point detected
# at index 84
## 84 84 TRUE changepoint

# Plot detection result vs. reference
har_plot(model, dataset$serie,
         detection, dataset$event)
```

Step by Step

- Load `examples_changepoints$simple`
- Plot the series to identify the labeled structural break
- Configure `hcp_amoc()` for single dominant break detection
- Run `detect()`: change-point found at index 84
- Plot detection vs. reference to compare results

📌 **Key concept:** AMOC (At Most One Change) scans every position and finds the split that best separates two statistically distinct regimes.

Smoothing with Moving Averages (MAS)

```
library(harbinger)

data(examples_changepoints)
dataset <- examples_changepoints$simple

# Plot the original signal
har_plot(harbinger(),
         dataset$serie,
         event = dataset$event)

# Compute moving averages with
# two different window sizes
ma_5 <- mas(dataset$serie, order = 5)
```

```
# Smaller window: retains more local
variation
har_plot(harbinger(),
         as.numeric(ma_5),
         event = dataset$event[
           5:length(dataset$event)])
ma_15 <- mas(dataset$serie, order = 15)

# Larger window: produces a smoother
series
har_plot(harbinger(),
         as.numeric(ma_15),
         event = dataset$event[
           15:length(dataset$event)])
```

Step by step

- Plot the original signal as a baseline
- `mas(order = 5)`: preserves local variation
- `mas(order = 15)`: yields a smoother series
- Adjust event vector index to match output length
- Compare plots to observe the smoothing trade-off

📌 **Key concept:** Moving averages suppress short-term noise by summarizing local neighborhoods, revealing low-frequency structure ideal for modeling or change-point detection.

Symbolic Representation of Time Series

SAX – Symbolic Aggregate approXimation

`alpha=8`, letters A–H. Divides value range into equiprobable Gaussian bins for compact subsequence comparison.

XSAX – Extended SAX

`alpha=16`, numbers 0–15. Finer-grained alphabet for capturing subtle variations and complex patterns.

- Converts continuous numeric values into discrete symbols
- Reduces dimensionality and suppresses noise while preserving shape
- Enables string-matching algorithms (suffix trees, edit distance) on time series
- In `harbinger`: `trans_sax()` and `trans_xsax()` follow the `fit()` → `transform()` pipeline from `daltoolbox`

Reference: Lin, J., Keogh, E., Wei, L., Lonardi, S. *Experiencing SAX: a novel symbolic representation of time series*. Data Mining and Knowledge Discovery, 2007.
doi:10.1007/s10618-007-0064-z

Symbolic Representation (SAX & XSAX)

```
library(daltoolbox)
library(harbinger)

data(examples_motifs)
dataset <- examples_motifs$simple

# Plot the signal before transforming
har_plot(harbinger(),
         dataset$serie,
         event = dataset$event)

# SAX: compact alphabet (8 symbols)
sax_model <- trans_sax(alpha = 8)
sax_model <- fit(sax_model, dataset$serie)
sax_series <- transform(sax_model,
                       dataset$serie)

head(sax_series, 5)
## [1] "D" "D" "D" "C" "C"
```

```
# XSAX: richer alphabet (16 symbols)
xsax_model <- trans_xsax(alpha = 16)
xsax_model <- fit(xsax_model, dataset$serie)
xsax_series <- transform(xsax_model,
                       dataset$serie)

head(xsax_series, 5)
## [1] "6" "6" "6" "5" "4"

# Compare both representations
head(data.frame(
  value = dataset$serie,
  sax   = sax_series,
  xsax  = xsax_series), 4)
## value sax xsax
## 1 1.000 D   6
## 2 0.994 D   6
## 3 0.928 D   6
## 4 0.807 C   5
```

Step by step

- Load `examples_motifs$simple` and plot the raw signal
- Fit `trans_sax(alpha = 8)` and apply the transformation
- Fit `trans_xsax(alpha = 16)` for a finer-grained alphabet
- Compare numeric value, SAX symbol, and XSAX symbol side by side
- SAX uses letters (A–H); XSAX uses numbers (0–15)

📌 **Key concept:** SAX and XSAX convert numeric values into discrete symbols, enabling easier subsequence comparison and pattern discovery at a higher level of abstraction.

Motif Discovery & Discord Detection

Motif

A recurring subsequence appearing multiple times with a similar shape.

Discord

The subsequence most unlike all others — the most anomalous in the series.

Used for pattern recognition, anomaly detection by contrast, and structural analysis of time series.

SAX-Based Motif Discovery Pipeline

- Discretize the series into symbols using `trans_sax()` or `trans_xsax()`
- Slide a window of length `window` across the symbolic series
- Flag subsequences appearing $\geq$ `min_rep` times as motifs
- `hmo_sax(alpha, window, min_rep)` — output columns: `seq` (motif group ID) and `seqlen` (window length)

Reference: Ogasawara, E., Salles, R., Porto, F., Pacitti, E. *Event Detection in Time Series*. Springer, 2025. doi:10.1007/978-3-031-75941-3

Motifs & Discords

```
library(daltoolbox)
library(harbinger)

data(examples_motifs)
dataset <- examples_motifs$simple

# Plot the series before searching for patterns
har_plot(harbinger(),
         dataset$serie,
         event = dataset$event)

# Configure the symbolic motif detector
# hmo_sax(alpha, window, min_rep)
model <- hmo_sax(8, 15, 3)
model <- fit(model, dataset$serie)
```

```
# Run detection
detection <- detect(model, dataset$serie)
head(detection)
## idx event type seq seqlen
## 1  1 FALSE NA    15
## 2  2 FALSE NA    15
## 3  3 FALSE NA    15

# Plot the motif-oriented result
har_plot(model, dataset$serie,
         detection, dataset$event)
```

Step by step

- Load `examples_motifs$simple` and plot the raw signal
- Configure `hmo_sax(alpha=8, window=15, min_rep=3)` for pattern search
- Fit and run `detect()` to find recurring subsequences
- Output includes `seq` and `seqlen` columns per window
- Plot results: recurring subsequences highlighted in context

📌 **Key concept:** Motif discovery asks *which subsequences repeat with similar shape*, using SAX symbolic encoding to compare windows efficiently.

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

